ADMM Research paper and implementation

— Prajith

Paper: Augmented Lagrangian Methods as Layered Control Architectures

—- (Anusha Srikanthan, Vijay Kumar, Nikolai Matni)

1. Intro:

- Usual control theory models: **trajectory generation layer** (to plan the nominal path) and a **feedback control layer** (to correct execution errors)

  Problem: It is imposed on the control policy beforehand (*a priori*) rather than naturally emerging from the mathematics of solving an optimal control problem

- Solution proposed: ADMM optimization algorithm :
  Using this, Naturally results in the layered structure (it mathematically links the trajectory and feedback layers using **Lagrange multipliers**)
- **Linear Systems:** Showing that for linear optimal control problems, this method yields an exact feedforward/feedback optimal controller, and can also incorporate stochastic process noise
- **Nonlinear Systems:** Proposing a heuristic algorithm for constrained nonlinear optimal control that uses iLQR (Iterative Linear Quadratic Regulator) as a subroutine, which empirically outperforms standard iLQR methods

2. Problem Formulation

- The optimal control problem (OCP) :
  Minimize $C_x(x) + C_u(u)$ where $x_{t+1}=f(x_t,u_t)$
  x: has to 1)obey physics 2)avoid obstacles
- Equivalent OCP
  Minimize $C_x(r) + C_u(u)$ where $x_{t+1}=f(x_t,u_t)$
  r: reference variable,deals with avoiding obstacles
  X:deals with physics
  r=x is the final goal
- Eg: for a drone, normal OCP's x: lots of params like x,y,z,vx,vy,vz,angles etc
  However in equivalent OCP, r=[x,y,z] {reals with collisions etc} and x contains remaining

2.1 Alternating direction method of multipliers (ADMM)

- Consider z : includes both x and u

$Ar+Bz=c$ corresponds to r=x

g(r) and h(z) are the cost functions

- Condition :minimize g(r) + h(z) subject to Ar + Bz = c

Equivalent constraint: (here the 2 conditions BOTH apply simultaneously)

$$L_\rho(r, z, v) = g(r) + h(z) + \frac{\rho}{2}\|Ar + Bz - c + v\|_2^2 - \frac{\rho}{2}\|v\|_2^2$$

- V: tracks how badly the constraint ($Ar+Bz=c$) is being violated.
  Rho: determines how harshly the algorithm penalizes constraint violations
- $r^{k+1} := \text{argmin}_r L_\rho(r, z^k, v^k)$ freezes z and v at their current values and finds the absolute best r to minimize the Lagrangian
- $z^{k+1} := \text{argmin}_z L_\rho(r^{k+1}, z, v^k)$
- $v^{k+1} := v^k + (Ar^{k+1} + Bz^{k+1} - c)$ updates v with current constraint error
- Convergence assumptions :
  1) G and h must be closed, proper, and convex
  2) Lagrangian must have a **saddle point**

     [r,z MINIMIZE cost , v MAXIMIZES penalty $\frac{\rho}{2}\|Ar + Bz - c + v\|_2^2$]

     [v's only job is to look at how badly r and z are cheating, and **maximize the penalty** for that specific violation]

If the assumptions satisfy, we get minimal cost and optimal value of v is reached

2.2 ADMM yields 2DOF layered control architectures in this OCP

- $Ar+Bz-c$ corresponds to r-x
- g(r) corresponds to $C_x(r)$
- h(z) = $C_u(u)$
- Lagrangian becomes: $C_x(r) + C_u(u) + \frac{\rho}{2}\|x - r + v\|_2^2 - \frac{\rho}{2}\|v\|_2^2$
- $r^{k+1} := \text{argmin}_r L_\rho(r, z^k, v^k)$,s.t. r ∈ R^N corresponds to $r^{k+1} := \text{argmin}_r \left(C_x(r) + \frac{\rho}{2}\|x^k - r + v^k\|_2^2\right)$ here (otherterms const)

  It minimizes $C_x(r)$, second term punishes if r is far from x

  Also r ∈ R^N tells to remain in safe region (without obstacles)

  Trajectory generation layer
- Similarly $(x^{k+1}, u^{k+1}) := \arg\min_{x,u} \left(C_u(u) + \frac{\rho}{2}\|x - r^{k+1} + v^k\|_2^2\right)$

  And $x_{t+1}=f(x_t,u_t)$

Feedback control layer

Minimizes $C_u(u)$ while we enforce **PHYSICS** $x_{t+1}=f(x_t,u_t)$

- $v^{k+1} := v^k + x^{k+1} - r^{k+1}$ If the planner creates a physically impossible path, x cannot perfectly follow r. The variable v absorbs this gap. In the *next* loop, v will alter the penalties, simultaneously forcing the planner to draw a more physically realistic path, and forcing the controller to track it more aggressively.

Dual update

Implementation

File structure:

admm.py :

Initializations:

```
r = x.copy()          # reference trajectory
r_target = np.tile([goal[0], goal[1], 0.0, 0.0, 0.0], N + 1)
x = np.tile(x0, N+1)  # state trajectory
size_x = n * (N + 1)
size_u = m * N
u = np.zeros(size_u)  # control sequence
v = np.zeros(size_x)  # dual variable
n=_N_STATES   = 5   # x, y, θ, v, ω
m=_N_CONTROLS = 2   # acceleration, steering rate (alpha)
```

V1 Update equations:(iterate till tol=Σ (r-x)2 is small)**

r update/ Trajectory generation layer

```
Lr(r) =L(x_, u_, r, v_, r_target, P, alpha) + obstacle_penalty(r, obstacles, n,
N)
res = minimize(Lr, x0=r_.copy())
r_ = res.x
```

Note: The condition : r ∈ R^N in the first update eq of ADMM is a hard constraint used in MILP

Since MILP isnt used here , we use a simple soft constraint of obstacle_penalty function [λ·penetration²]with weight=10000, the penalty is so steep that the optimiser almost never accepts solutions that penetrate obstacles.

X,u update / Feedback control layer

Method used : SLSQP : Sequential Least Squares Quadratic Programming. It's a gradient-based algorithm for solving nonlinear programs of the form “minimize f(z) subject to some (equality constraints) and (inequality constraints)”

```
 cons = build_dynamics_constraint(x0, n, m, N, dt)[in dynamics.py]
Lxu(z)= L(xz, uz, r_, v_, r_target, P, alpha)
res = minimize(Lxu, x0=z0, constraints=cons, method="SLSQP")
```

```
x_  = res.x[:size_x]
u_  = res.x[size_x:]
```

V update:

```
v_ += x_ - r_
```

V2 Update Equations:

r update/ Trajectory generation layer

```
Lr(r) =L(x_, u_, r, v_, r_target, P, alpha) + obstacle_penalty(r, obstacles, n, N)

Lr_grad(r) =Cx_grad(r, r_target, alpha) - P *(x_ - r + v_ ) + obstacle_penalty_grad(r, obstacles, n, N)

res = minimize(Lr, x0=r_.copy(), jac=Lr_grad, method="L-BFGS-B")
r_ = res.x
```

Explanation:

- Lr_grad(r) is the derivative of Lr(r) wrt r
- jac=Lr_grad : It tells Scipy to take the gradient directly instead of estimating it [increases computation speed]
- L-BFGS-B method: No equality constraints here ,Unconstrained so L-BFGS-B is the right tool

  L-BFGS-B cannot accept equality constraints while SLSQP can accept

  Also The Obstacle penalty term makes Lr non convex

  L-BFGS-B method is used as it makes no convexity assumption

X,u update / Feedback control layer

Method used :

```
Lxu(z)= L(xz, uz, r_, v_, r_target, P, alpha)
Lxu_grad(z) :  [g_x = P * (xz - r_ + v_) ,g_u = 2.0 * (1.0 - alpha) * uz ]
res = minimize(Lxu, x0=z0, jac=Lxu_grad, constraints=cons, method="SLSQP")
x_  = res.x[:size_x]
u_  = res.x[size_x:]
```

Note: cons here is updated build_dynamics_constraint function

Explanation:

SLSQP's job at each iteration is to solve a local Quadratic Programming (QP) sub-problem to find a step direction δz. That sub-problem has two separate ingredients:

“Minimize” → needs Lxu_grad

“subject to cons” → needs dynamics_jac from build_dynamics_constraint [check dynamics.py section]

Partial derivative wrt x and wrt u gives Lxu_grad

Equality constraints so SLSQP applicable

```
v_ += x_ - r_
```

costs.py:

Implementing the Lagrangian:

```
def L(x, u, r, v, r_target, P, alpha):
    return (Cx(r, r_target, alpha)+ Cu(u, alpha)+ (P / 2) * (np.linalg.norm(x -
r + v) ** 2- np.linalg.norm(v)       ** 2))
```

V1 implementation of cost functions/lagrangians:

Cx(r, r_target, alpha) : $C_x(r) = \alpha \cdot \|r - r^*\|^2$

Cu(u, alpha) : $C_u(u) = (1 - \alpha) \cdot \|u\|^2$

```
α = weight_ratio / (weight_ratio + 1)   where weight_ratio is user input
```

V2 implementation of cost functions/lagrangians:

Cx(r, r_target, alpha) : $\alpha * \Sigma\, w_t * (r_t - r_target_t)^2$

_weights(r) : _STAGE_W*1 for states 1 to N-1 and

_TERMINAL_W*_TERMINAL_SCALE for Nth state

Cx_grad(r, r_target, alpha) : 2*α*_weights(r) * (r-r_target)

Explanation:

- Cx_grad : Partial derivative wrt r_i then combine vectorially
- It pulls harder on the terminal state than on intermediate state

Cu(u) : $\Sigma\ (1-\alpha) * \|u\|^2$

```
_STAGE_W = np.array([1.0, 1.0, 0.0, 0.0, 0.0])       # penalise x,y only
```

_TERMINAL_W = np.array([1.0, 1.0, 0.0, 0.3, 0.3]) # also penalise v,ω at end

```
_TERMINAL_SCALE = 80.0
```

Explanation: At every intermediate timestep, only position errors (x_1, x_2) are penalised; θ, v, and ω are free. At the final timestep, velocity and angular rate are also penalised (scaled ×80), encouraging the robot to come to rest at the goal rather than just pass through it.

Θ: doesnt really matter as directions dont make sense when the goal is to reach the goal

dynamics.py:

```
unicycle_step(state, ctrl, dt) :  [x1, x2, theta, v, omega = state a, alpha_acc
= ctrl ]
```

This is the f(x_t, u_t) function in the paper

```
x1 + v * np.cos(theta) * dt, # position x
x2 + v * np.sin(theta) * dt, # position y
theta + omega * dt, # heading
```

```
v + a * dt, # velocity (integrated from acceleration)
omega + alpha_acc * dt, # angular rate (integrated from angular acc) ])
```

V1 function f:

```
build_dynamics_constraint(x0_state, n, m, N, dt)
```

Implements x_next = unicycle_step(xt,ut, dt) through timestamps

This is the CONSTRAINT (f) used in the X,U update layer

V2 function f modelling:

```
unicycle_step_jac(state,ctrl,dt):
```

Finds Jx=df/dx and Ju=df/du matrices

	X1	x2	theta	v	ω
F1	1.0	0.0	-v * np.sin(theta) * dt	np.cos(theta) * dt	0.0
F2	0.0	1.0	v * np.cos(theta) * dt	np.sin(theta) * dt	0.0
F3	0.0	0.0	1.0	0.0	dt
F4	0.0	0.0	0.0	1.0	0.0
F5	0.0	0.0	0.0	0.0	1.0

$f_1(x,u) = x_1 + v \cdot \cos(\theta) \cdot dt$

$f_2(x,u) = x_2 + v \cdot \sin(\theta) \cdot dt$

$f_3(x,u) = \theta + \omega \cdot dt$

$f_4(x,u) = v + a \cdot dt$

$f_5(x,u) = \omega + \alpha_acc \cdot dt$

	a	α
F1	0.0	0.0
F2	0.0	0.0
	0.0	0.0
	dt	0.0
	0.0	dt

Only the velocity/angular-rate rows are nonzero in Ju, since only those states are directly driven by the controls.

Updated Build_dynamics_constraint function:(contains dynamics_jac function inbuilt)

Residuals records “by how much is this constraint violated?”

```
residual_t = x_{t+1} - f(x_t, u_t)      for all time stamps
```

- If physics is perfectly satisfied: residual_t = 0 for all t

  If physics is violated: residual_t ≠ 0, and the magnitude tells you how bad the violation is
- Residuals vector contains the residuals from t=0 to t-N-1

  When this vector is 0, z = (x,u) describes a valid trajectory
- **Dynamics_jac function**

  SLSQP needs to know how the residuals change when you nudge z.

  This is the Jacobian matrix:

```
J[i, j]  =  d(residuals[i]) / d(z[j])
```

```
z = [ x₀ | x₁ | x₂ | ... | x□ | u₀ | u₁ | ... | u_{N-1} ]
residuals = [ r₀ | r₁ | r₂ | ... | r□ ]
```

```
dr_{t+1}/d(x_{t+1}) = +I_n      (nudging x_{t+1} directly changes r_{t+1})
dr_{t+1}/d(x_t)     = −Jx       (nudging x_t changes f(x_t,u_t), which affects
r_{t+1})
dr_{t+1}/d(u_t)     = −Ju       (nudging u_t changes f(x_t,u_t), which affects
r_{t+1})
dr_{t+1}/d(anything else) = 0
```

The full Jacobian matrix looks like this :

Columns:	x_0	x_1	x_2	x_3	u_0	u_1	u_2
r_0 rows	I						
r_1 rows	$-Jx_0$	I			$-Ju_0$		
r_2 rows		$-Jx_1$	I			$-Ju_1$	
r_3 rows			$-Jx_2$	I			$-Ju_2$

The function returns dynamics and dynamics_jac

constraints.py:

```
obstacle_penalty( r_flat, obstacles, n, N, weight=10000.0):
```

Rectangle obstacles :when the reference point (rx, ry) is inside a rectangle, the penalty grows quadratically with the depth of penetration (distance to the nearest wall).

```
depth = min(rx - x_min, x_max - rx,ry - y_min, y_max - ry,)
penalty += weight * depth ** 2
```

Circle obstacles: the penalty grows quadratically with how far inside the circle the reference point has moved.

```
dist = np.sqrt((rx - cx) ** 2 + (ry - cy) ** 2) + 1e-9
penetration = max(0.0, radius - dist)
penalty += weight * penetration ** 2
```

Final sol also implements:

```
obstacle_penalty_grad( )
```

Explanation:

Calculates d penalty/rx and ry case by case basis

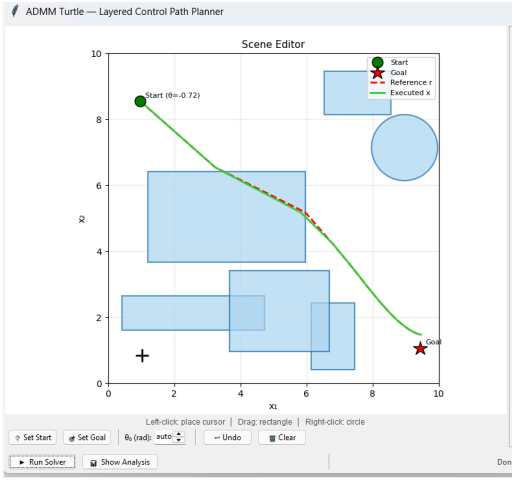
Note:the paper uses MILP approach

Property	Paper (MILP)	Code (Soft Penalty)
Obstacle avoidance guarantee	Hard (exact)	Soft (probabilistic and depends on `weight`)
Solver type required	MILP (cvxpy req)	Any gradient-based solver
Scales with obstacles	Exponentially (integer vars)	Linearly

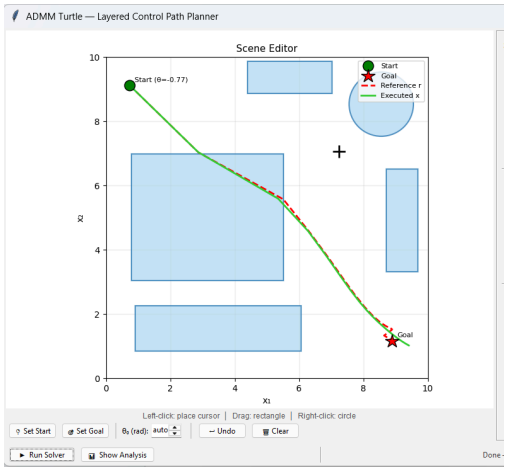
Is any possible way of extending to any obstacle in code sol?

Yes : Signed Distance Function (SDF)

Implementation(V2) Observations



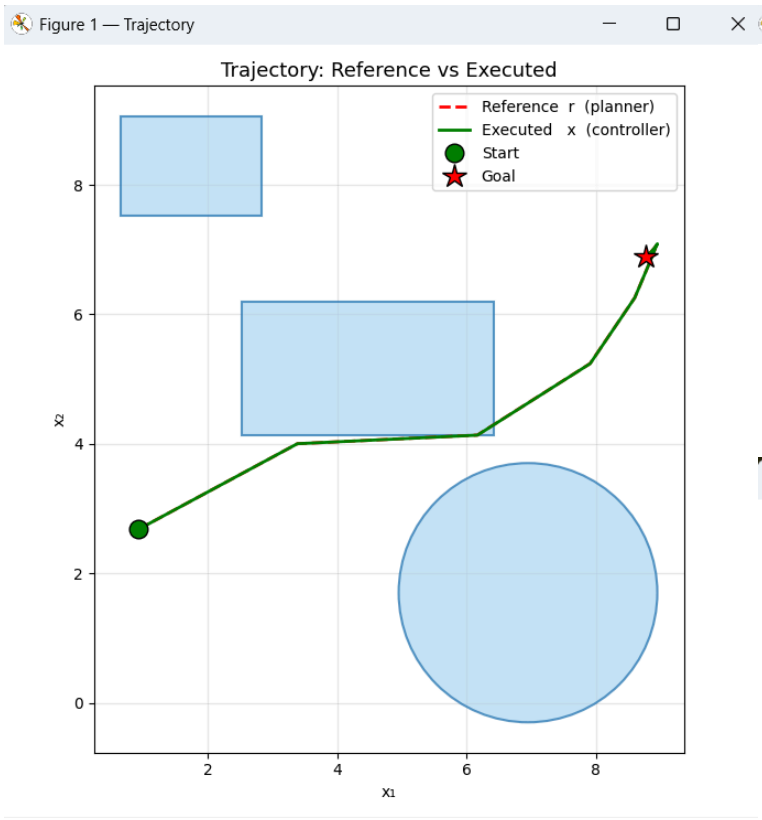
Without Terminal weights

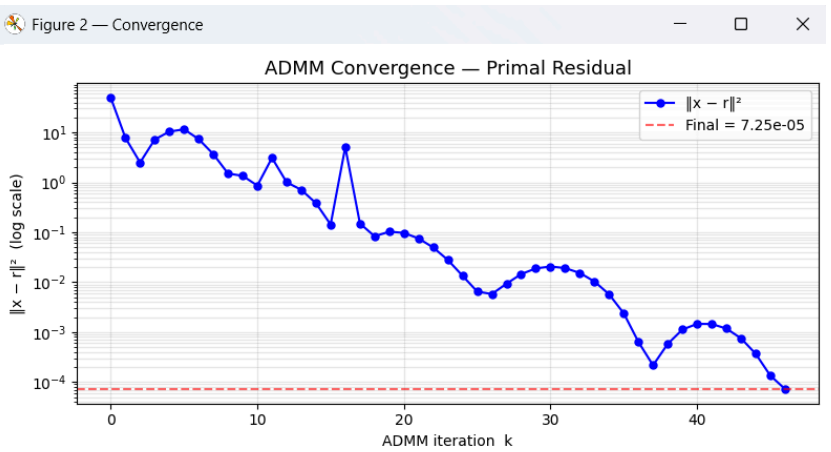


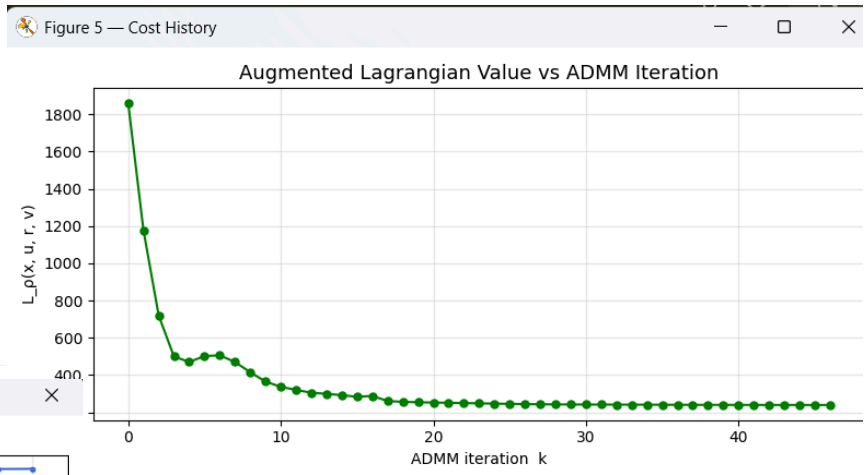
With Terminal Weights

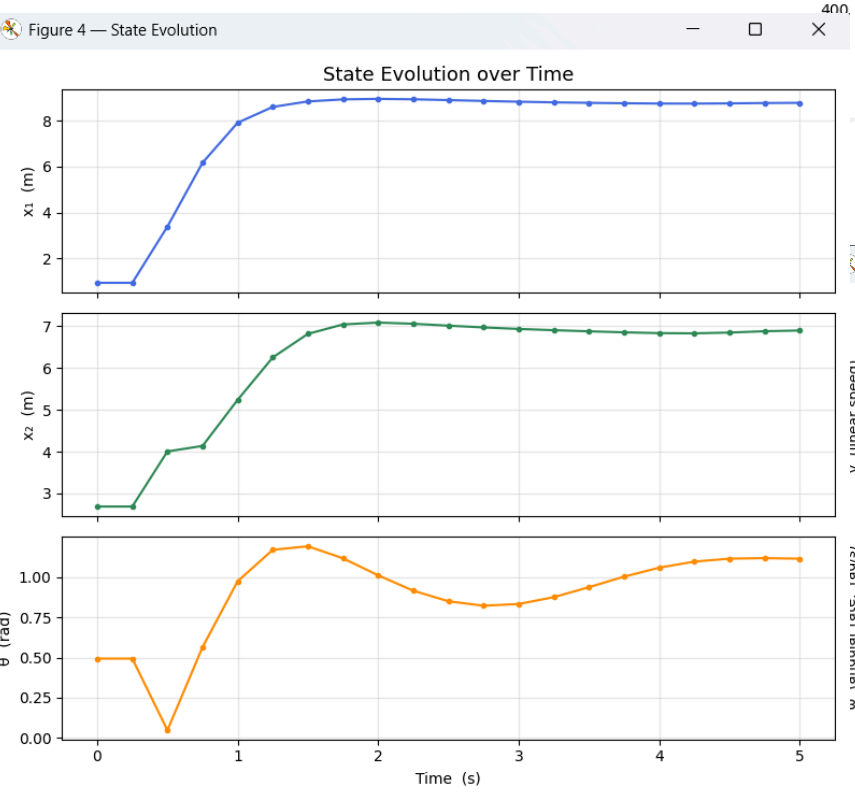
Sample Implementation

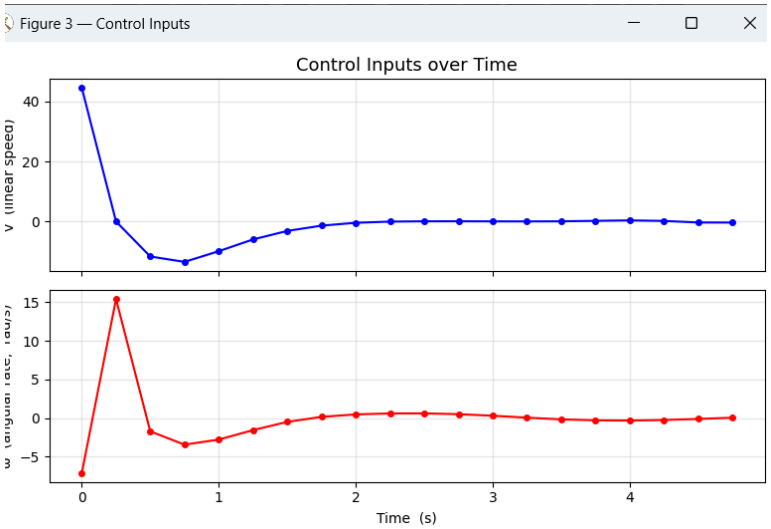
Analysis graphs:











Limitations of ADMM:

- **No convergence guarantee for nonlinear problems:**

  ADMM's convergence theorem strictly requires both cost functions to be **closed, proper, and convex** .

  The unicycle dynamics are nonlinear and the obstacle penalties are non-convex. The paper itself explicitly calls the nonlinear version a *"heuristic algorithm"*. It can get stuck in local minima, oscillate, or fail to converge entirely , there is no mathematical guarantee it will reach the correct solution.

- **Fixed horizon myopia**

  The planner optimises over exactly N timesteps.

  If navigating around a large obstacle requires a detour that takes more than N steps to complete, the solver cannot see the goal past the obstacle

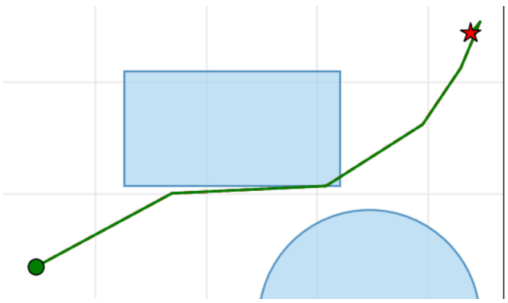
  The solver then tries to go through the obstacle because that's the only path it can see within N steps.

- **Soft penalty gradient weakness at obstacle boundaries**

  The rectangle penalty uses `depth = min(rx - x_min, x_max - rx, ry - y_min, y_max - ry)`. At the very edge, depth ⟶ 0, so the penalty ⟶ 0 and its gradient is nearly zero.

  The L-BFGS-B optimizer gets essentially no signal that it's about to cross the boundary, so it drifts through.

  Additionally, the `min()` function is non-smooth at obstacle corners, which violates the smoothness assumption L-BFGS-B relies on — the optimizer can take bad steps near corners.



——————> soft obstacle violation